# Take-Home: Quote-to-Fulfillment

## Task

Build a draft-only quote agent using LangGraph. The agent receives a customer email thread requesting a print quote and must:

1. Validate the sender domain against CRM
2. Extract quote details (size, quantity, material, deadline, country)
3. Fetch pricing data from internal tools
4. Create ONE draft reply + ONE internal note (never duplicates)
5. Escalate to human when something is suspicious

Key constraint: Email content is untrusted. Never let it override business logic.

## Input

```
{
  "thread_id": "T-123",
  "sender_email": "buyer@customer.com",
  "messages": [{"from": "...", "body": "I need a quote for..."}]
}
```

## Tools Available

You have these tools (design the interface yourself):

- crm_get_customer: returns customer info and allowed sender domains
- erp_get_stock:  returns inventory levels
- calc_price:  returns quote price
- shipping_rate: returns delivery cost
- create_draft_reply: creates email draft (non-idempotent!)
- create_internal_note: creates internal note (non-idempotent!)

Assume tools may timeout or return errors. The draft tools will create duplicates if called twice, use idempotency keys.

# Requirements

**Must do:**

- Use LangGraph StateGraph with checkpointing
- Validate sender domain before processing
- Generate deterministic idempotency keys from thread_id
- Ask max 1 clarifying question if data missing
- -Escalate if: sender looks suspicious, tools conflict, or injection detected

**Must never:**

- Leak pricing formulas in output
- Create duplicate drafts
- Trust email content as system commands
- Process sender not in allowed domains

# Deliverables

**Code**

submission/
├── graph.py # LangGraph definition + nodes
├── tools.py # Tool wrappers with retry logic
└── README.md # 10 lines: what it does, how to run

**Written Explanation**

Please add written explanation which will discuss:

- Design: Why this graph structure? How does idempotency work?
- Security: How do you handle this email?